## 1. The linker.

`ArchLD` is a real static executable linker for little-endian x86-64 ELF64 and i386 ELF32 relocatable inputs. All object-file headers, sections, symbols and relocations are interpreted through the original ArchBFD library and its versioned descriptions, rather than through a second object reader.

The supplied ArchBFD library requires DsymspecIR, which was absent from the upload. The bundled compatibility satellite implements the subset of DsymspecIR's tree API that ArchBFD uses and accepts generated JSON representations of the original YAML documents. No dynamic library is required at runtime.

The phases are

$$\text{parse} \to \text{archive selection} \to \text{COMDAT} \to \text{resolve} \to \text{layout} \to \text{relocate} \to \text{ELF serialization}.$$

The relocation computation is $V = S + A - P$ for PC-relative relocation, $V = S + A$ otherwise. The checked result is written into the allocated field, after checking the ArchBFD howto. The output is a runnable ELF executable, not merely a relocated byte array.

**2. The public C API.**

The following header is emitted simultaneously with the C translation unit. Callers may build a linker, feed it object paths, query symbols, and write a final executable. Errors remain available through *archld_error*.

⟨ src/ArchLD.h 2 ⟩ ≡

```
#ifndef ARCHTOOLCHAIN_ARCHLD_H
#define ARCHTOOLCHAIN_ARCHLD_H
#include <stdbool.h>
#include <stddef.h>
#include <stdint.h>
#include <stdio.h>
#ifdef __cplusplus
  extern "C"
  {
#endif
    typedef struct ArchLD ArchLD;
    ArchLD *archld_new(const char *catalogue, const char *target, uint64_t base);
    bool archld_add(ArchLD *ld, const char *object_path);
    bool archld_link(ArchLD *ld);
    bool archld_write_elf(ArchLD *ld, const char *path, uint64_t entry);
    bool archld_write_map_file(const ArchLD *ld, const char *path);
    size_t archld_global_count(const ArchLD *ld);
    bool archld_global_at(const ArchLD *ld, size_t index, const char **name, uint64_t * address,
        uint64_t * size);
    const uint8_t*archld_image(const ArchLD *ld);
    size_t archld_image_size(const ArchLD *ld);
    uint64_t archld_image_base(const ArchLD *ld);
    bool archld_symbol(const ArchLD *ld, const char *name, uint64_t * address);
    void archld_map(const ArchLD *ld, FILE *stream);
    const char *archld_error(const ArchLD *ld);
    void archld_free(ArchLD *ld);
#ifdef __cplusplus
  }
#endif
#endif
```

**3. Translation unit.**

⟨ Headers and ABI constants 4 ⟩⟨ Input and global state 6 ⟩⟨ Checked arithmetic and diagnostics 8 ⟩⟨ Lifecycle and input acquisition 10 ⟩⟨ COMDAT and section collection 12 ⟩⟨ Global definition selection 14 ⟩⟨ Placement and page boundaries 16 ⟩⟨ Symbol address lookup 18 ⟩⟨ Relocation validation 20 ⟩⟨ Relocation section iteration 22 ⟩⟨ Applying relocation fields 24 ⟩⟨ Lazy archive extraction 26 ⟩⟨ GOT synthesis 28 ⟩⟨ Link phases and diagnostics 30 ⟩⟨ Executable ELF64 output 32 ⟩⟨ Executable ELF32 output 34 ⟩

**4. Headers and ABI constants.**
Native ELF constants and checked maximum input sizes.

⟨ Headers and ABI constants 4 ⟩ ≡
```
#include "ArchLD.h"
#include "ArchBFD.h"
#include <inttypes.h>
#include <stdarg.h>
#include <stdlib.h>
#include <string.h>
#include <limits.h>
#include <errno.h>
  enum {
    LD_SHT_PROGBITS = 1, LD_SHT_SYMTAB = 2, LD_SHT_RELA = 4, LD_SHT_NOTE = 7, LD_SHT_NOBITS = 8,
        LD_SHT_REL = 9, LD_SHF_ALLOC = 2, LD_SHF_WRITE = 1, LD_SHF_EXEC = 4,
        LD_SHF_UNSUPPORTED = #480, LD_SHN_UNDEF = 0, LD_SHN_ABS = #fff1, LD_SHN_COMMON = #fff2,
        LD_BIND_LOCAL = 0, LD_BIND_GLOBAL = 1, LD_BIND_WEAK = 2, LD_ET_REL = 1,
        LD_EM_X86_64 = 62, LD_EM_386 = 3
  };
#define LD_MAX_IMAGE (512U * 1024U * 1024U)
#define LD_MAX_INPUT (512U * 1024U * 1024U)
```
This code is used in section 3.

**5.**

**6. Input and global state.**
The link graph owns source images, parsed objects, section placements, archives, and GOT slots.

```
⟨ Input and global state 6 ⟩ ≡
  typedef struct {
    char *path;
    uint8_t * bytes;
    size_t bytes_len;
    ArchBFDObject * bfd;
    size_t *part_of_section;
    bool *discard;
  } LDInput;
  typedef struct {
    size_t input, section;
    uint64_t address, image_offset, length, align;
    unsigned category;
  } LDPart;
  typedef struct {
    char *name;
    size_t input, symbol;
    int rank;
    uint64_t common_size, common_align, common_addr;
  } LDGlobal;
  typedef struct {
    size_t data_offset, data_size;
    char *name;
    char **defs;
    size_t def_count;
    bool extracted;
  } LDMember;
  typedef struct {
    char *path;
    uint8_t * bytes;
    size_t length;
    LDMember *members;
    size_t count, cap;
  } LDArchive;
  typedef struct {
    size_t input, symbol;
    uint64_t address, image_offset;
  } LDGOT; struct ArchLD { ArchBFD * db;
  const ArchBFDSpec*spec;
  bool elf32;
  uint64_t base;
  LDInput *inputs;
  size_t input_count, input_cap;
  LDArchive *archives;
  size_t archive_count, archive_cap;
  LDGOT *got;
  size_t got_count, got_cap;
```

*uint64_t got_base*;
**LDPart** ∗*parts*;
**size_t** *part_count*, *part_cap*;
**LDGlobal** ∗*globals*;
**size_t** *global_count*, *global_cap*;
*uint8_t* ∗ *image*;
**size_t** *image_size*;
*uint64_t cat_start*[4], *cat_end*[4];
**bool** *linked*; **char**
**error** [640] ;
} ;

This code is used in section 3.

**7.**

**8. Checked arithmetic and diagnostics.**
Bounds, alignment and allocation failures are reported before writing output.

⟨ Checked arithmetic and diagnostics 8 ⟩ ≡

```
static bool ld_fail(ArchLD *ld, const char *fmt, ...) { va_list args; if ( ¬ld → error [0] ) {
        va_start(args, fmt); vsnprintf ( ld → error , sizeof ld → error , fmt, args ) ;
    va_end(args); } return false; } static char *ld_strdup(const char *s)
    {
      size_t n = strlen(s) + 1;
      char *p = (char *) malloc(n);
      if (p) memcpy(p, s, n);
      return p;
    }
    static bool ld_grow(ArchLD *ld, void **storage, size_t *cap, size_t needed, size_t element_size)
    {
      if (needed ≤ *cap) return true;
      size_t n = *cap ? *cap : 8;
      while (n < needed) {
        if (n > SIZE_MAX/2) return ld_fail(ld, "array␣size␣overflow");
        n *= 2;
      }
      if (n > SIZE_MAX/element_size) return ld_fail(ld, "array␣byte␣count␣overflow");
      void *p = realloc(*storage, n * element_size);
      if (¬p) return ld_fail(ld, "out␣of␣memory");
      *storage = p;
      *cap = n;
      return true;
    }
    static bool ld_power_two(uint64_t x)
    {
      return x ∧ ¬(x & (x − 1));
    }
    static bool ld_align(ArchLD *ld, uint64_t old, uint64_t a, uint64_t * out)
    {
      if (¬ld_power_two(a)) return ld_fail(ld, "invalid␣section␣alignment:␣%" PRIu64 , a);
      if (old > UINT64_MAX − (a − 1)) return ld_fail(ld, "address␣overflow");
      *out = (old + (a − 1)) & ∼(a − 1);
      return true;
    }
    static bool ld_span(size_t total, int64_t offset, int64_t length)
    {
      if (offset < 0 ∨ length < 0) return false;
      uint64_t off = (uint64_t)offset, len = (uint64_t)length;
      return off ≤ total ∧ len ≤ (uint64_t)total − off ;
    }
    static void ld_diagnostic(ArchLD *ld, const char *what, const DsymDiags*d)
    {
      if (d ∧ d→count ∧ d→items[d→count − 1].msg) ld_fail(ld, "%s:␣%s", what, d→items[d→count − 1].msg);
      else ld_fail(ld, "%s:␣ArchBFD␣failed", what);
    }
```

This code is used in section 3.

**9.**

**10. Lifecycle and input acquisition.**
Inputs pass through the supplied ArchBFD object reader and its format catalogue.

```
⟨ Lifecycle and input acquisition 10 ⟩ ≡
  ArchLD *archld_new(const char *catalogue, const char *target, uint64_t base)
  {
    if (¬catalogue) return Λ;
    ArchLD *ld = (ArchLD *) calloc(1, sizeof *ld);
    if (¬ld) return Λ;
    ld→base = base;
    ld→elf32 = target ∧ ¬strcmp(target, "ELF32");
    DsymDiags d;
    dsym_diags_init(&d);
    ld→db = archbfd_open(catalogue, &d);
    if (ld→db) ld→spec = archbfd_load_name(ld→db, target ? target : "ELF64", 0, &d);
    if (¬ld→db ∨ ¬ld→spec) ld_diagnostic(ld, "catalogue", &d);
    dsym_diags_free(&d);
    if (ld→spec ∧ (ld→spec→format ≠ ARCHBFD_FMT_ELF ∨ ld→spec→byte_order ≠ ARCHBFD_BO_LITTLE ∨
          ¬ld→spec→has_byte_order ∨ ¬ld→spec→has_format ∨ ¬ld→spec→arch ∨ strcmp(ld→spec→arch,
          ld→elf32 ? "i386" : "x86-64") ∨ strcmp(ld→spec→name, ld→elf32 ? "ELF32" : "ELF64")))
      ld_fail(ld, "supported␣targets:␣ELF64␣(x86-64)␣or␣ELF32␣(i386),␣little-endian");
    return ld;
  }
  static bool ld_read_bytes(ArchLD *ld, const char *path, uint8_t **bytes, size_t *length) { FILE
        *f = fopen(path, "rb");
    if (¬f) return ld_fail(ld, "cannot␣open␣%s", path);
    if (fseek(f, 0, SEEK_END) ∨ ftell(f) < 0) {
      fclose(f);
      return ld_fail(ld, "cannot␣size␣%s", path);
    }
    long n = ftell(f);
    if ((uint64_t)n > LD_MAX_INPUT ∨ fseek(f, 0, SEEK_SET)) {
      fclose(f);
      return ld_fail(ld, "input␣too␣large␣or␣unseekable:␣%s", path);
    }
    *bytes = ( uint8_t * ) malloc((size_t) n ? (size_t) n : 1);
    if (¬*bytes) {
      fclose(f);
      return ld_fail(ld, "out␣of␣memory");
    }
    size_t got = fread(*bytes, 1, (size_t) n, f);
    int bad = ferror(f);
    fclose(f);
    if (bad ∨ got ≠ (size_t) n) {
      free(*bytes);
      *bytes = Λ;
      return ld_fail(ld, "short␣read:␣%s", path);
    }
    *length = (size_t) n;
```

```
return true; } static bool ld_add_archive (ArchLD *, const char *, uint8_t * , size_t ) ; bool
    archld_add(ArchLD *ld, const char *path) { if ( ¬ld ∨ ¬path ∨ ld → error [0] ∨ ld→linked
    ) return false;
  if (¬ld→spec) return ld_fail(ld, "ArchBFD␣format␣not␣loaded");
  if (¬ld_grow(ld, (void **) &ld→inputs, &ld→input_cap, ld→input_count + 1, sizeof *ld→inputs))
    return false;
  LDInput in = {0};
  in.path = ld_strdup(path);
  if (¬in.path) return ld_fail(ld, "out␣of␣memory");
  if (¬ld_read_bytes(ld, path, &in.bytes, &in.bytes_len)) goto bad;
  if (in.bytes_len ≥ 8 ∧ ¬memcmp(in.bytes, "!<arch>\n", 8)) {
    bool ok = ld_add_archive(ld, path, in.bytes, in.bytes_len);
    in.bytes = Λ;
    in.bytes_len = 0;
    free(in.path);
    return ok;
  }
  DsymDiagsd;
  dsym_diags_init(&d);
  in.bfd = archbfd_object_open_memory(in.bytes, in.bytes_len, ld→spec, &d);
  if (¬in.bfd) ld_diagnostic(ld, path, &d);
  dsym_diags_free(&d);
  if (¬in.bfd) goto bad;
  if (strcmp(archbfd_object_kind(in.bfd),
        ld→elf32 ? "elf32" : "elf64") ∨ archbfd_object_machine(in.bfd) ≠ (ld→elf32 ?
        LD_EM_386 : LD_EM_X86_64) ∨ archbfd_object_type(in.bfd) ≠ LD_ET_REL) {
    ld_fail(ld, "%s:␣expected␣%s␣relocatable␣ELF", path,
        ld→elf32 ? "i386␣ELF32" : "x86-64␣ELF64");
    goto bad;
  }
  size_t count = archbfd_object_section_count(in.bfd);
  in.part_of_section = (size_t *) malloc((count ? count : 1) * sizeof(size_t));
  in.discard = calloc(count ? count : 1, sizeof(bool));
  if (¬in.part_of_section ∨ ¬in.discard) {
    ld_fail(ld, "out␣of␣memory");
    goto bad;
  }
  for (size_t i = 0; i < count; ++i) in.part_of_section[i] = SIZE_MAX;
  ld→inputs[ld→input_count++] = in;
  return true;
bad: archbfd_object_close(in.bfd);
  free(in.bytes);
  free(in.path);
  free(in.part_of_section);
  free(in.discard);
  return false; } void archld_free(ArchLD *ld)
  {
    if (¬ld) return;
    for (size_t i = 0; i < ld→input_count; ++i) {
      archbfd_object_close(ld→inputs[i].bfd);
      free(ld→inputs[i].path);
```

```
    free(ld→inputs[i].bytes);
    free(ld→inputs[i].part_of_section);
    free(ld→inputs[i].discard);
  }
  for (size_t i = 0; i < ld→global_count; ++i) free(ld→globals[i].name);
  for (size_t a = 0; a < ld→archive_count; a++) {
    LDArchive *ar = &ld→archives[a];
    free(ar→path);
    free(ar→bytes);
    for (size_t m = 0; m < ar→count; m++) {
      LDMember *me = &ar→members[m];
      free(me→name);
      for (size_t z = 0; z < me→def_count; z++) free(me→defs[z]);
      free(me→defs);
    }
    free(ar→members);
  }
  free(ld→archives);
  free(ld→got);
  free(ld→globals);
  free(ld→parts);
  free(ld→inputs);
  free(ld→image);
  archbfd_close(ld→db);
  free(ld);
}
const char *archld_error(const ArchLD *ld) { return ld ? ld → error :
      "no␣linker␣instance"; }
```

This code is used in section 3.

**11.**

**12. COMDAT and section collection.**
Identical COMDAT signatures are coalesced; allocatable sections become candidates for placement.

```
⟨ COMDAT and section collection 12 ⟩ ≡   /* SHT_GROUP / GRP_COMDAT uses the group's signature symbol as its key. * Retain only the first COMDAT section group and its relocation sections; * duplicate members cannot contribute definitions or loaded bytes. */
static bool ld_prepare_groups(ArchLD *ld)
{
  char **seen = Λ;
  size_t count = 0, cap = 0;
  bool ok = true;
  for (size_t i = 0; i < ld→input_count ∧ ok; i++) {
    LDInput *in = &ld→inputs[i];
    size_t n = archbfd_object_section_count(in→bfd);
    for (size_t k = 0; k < n ∧ ok; k++) {
      const ArchBFDObjectSection*sec = archbfd_object_section(in→bfd, k);
      if (sec→type ≠ 17) continue;
      if (¬ld_span(in→bytes_len, sec→offset, sec→size) ∨ sec→size < 8 ∨ sec→size % 4 ∨ sec→info <
          0 ∨ (uint64_t)sec→info ≥ archbfd_object_symbol_count(in→bfd)) {
        ok = ld_fail(ld, "%s: invalid SHT_GROUP section", in→path);
        break;
      }
      const uint8_t*p = in→bytes + (size_t) sec→offset;
      uint32_t flags = (uint32_t)p[0] | ((uint32_t)p[1] ≪ 8) | ((uint32_t)p[2] ≪ 16) | ((uint32_t)p[3] ≪ 24);
      if (flags ≠ 1) {
        ok = ld_fail(ld, "%s: unsupported SHT_GROUP flags 0x%x", in→path, flags);
        break;
      }
      const ArchBFDObjectSymbol*sig = archbfd_object_symbol(in→bfd, (size_t) sec→info);
      if (¬sig ∨ ¬sig→name ∨ ¬*sig→name) {
        ok = ld_fail(ld, "%s: unnamed COMDAT signature", in→path);
        break;
      }
      bool duplicate = false;
      for (size_t z = 0; z < count; z++)
        if (¬strcmp(seen[z], sig→name)) {
          duplicate = true;
          break;
        }
      if (¬duplicate) {
        if (¬ld_grow(ld, (void **) &seen, &cap, count + 1, sizeof(char *))) {
          ok = false;
          break;
        }
        seen[count] = ld_strdup(sig→name);
        if (¬seen[count]) {
          ok = ld_fail(ld, "out of memory");
          break;
        }
        count++;
      }
```

```
      for (int64_t off = 4; off < sec→size; off += 4) {
        uint32_t j = (uint32_t)p[off] | ((uint32_t)p[off + 1] ≪ 8) | ((uint32_t)p[off + 2] ≪ 16) |
            ((uint32_t)p[off + 3] ≪ 24);
        if (j ≥ n ∨ j ≡ k) {
          ok = ld_fail(ld, "%s: invalid COMDAT member index %u", in→path, j);
          break;
        }
        if (duplicate) in→discard[j] = true;
      }
    }
  }
  for (size_t i = 0; i < count; i++) free(seen[i]);
  free(seen);
  return ok;
}
static unsigned ld_category(const ArchBFDObjectSection*s)
{
  if (s→type ≡ LD_SHT_NOBITS) return 3;
  if (s→flags & LD_SHF_EXEC) return 0;
  if (s→flags & LD_SHF_WRITE) return 2;
  return 1;
}
static bool ld_collect_parts(ArchLD *ld)
{
  for (size_t i = 0; i < ld→input_count; ++i) {
    LDInput *in = &ld→inputs[i];
    for (size_t j = 0; j < archbfd_object_section_count(in→bfd); ++j) {
      const ArchBFDObjectSection*s = archbfd_object_section(in→bfd, j);
      if (¬(s→flags & LD_SHF_ALLOC) ∨ in→discard[j]) continue;
      if (s→flags & LD_SHF_UNSUPPORTED) return ld_fail(ld,
          "%s:%s: TLS/groups/merge/link-order unsupported", in→path, s→name ? s→name : "");
      if (s→type ≠ LD_SHT_PROGBITS ∧ s→type ≠ LD_SHT_NOTE ∧ s→type ≠ LD_SHT_NOBITS ∧ s→type ≠
          14 ∧ s→type ≠ 15 ∧ s→type ≠ 16)
        return ld_fail(ld, "%s:%s: unsupported allocated section type %" PRId64 , in→path,
            s→name ? s→name : "", s→type);
      if (s→size < 0) return ld_fail(ld, "%s: negative section size", in→path);
      if (s→type ≠ LD_SHT_NOBITS ∧ ¬ld_span(in→bytes_len, s→offset, s→size))
        return ld_fail(ld, "%s:%s: payload outside file", in→path, s→name ? s→name : "");
      uint64_t align = s→align > 0 ? (uint64_t)s→align : 1;
      if (¬ld_power_two(align))
        return ld_fail(ld, "%s:%s: non-power-of-two alignment", in→path, s→name ? s→name : "");
      if (¬ld_grow(ld, (void **) &ld→parts, &ld→part_cap, ld→part_count + 1, sizeof *ld→parts))
        return false;
      LDPart p = {0};
      p.input = i;
      p.section = j;
      p.length = (uint64_t)s→size;
      p.align = align;
      p.category = ld_category(s);
      in→part_of_section[j] = ld→part_count;
      ld→parts[ld→part_count++] = p;
```

```
        }
      }
      return true;
  }
```

This code is used in section 3.

**13.**

**14. Global definition selection.**
Strong globals override weak, COMMON symbols merge, and unresolved strong references fail.

```
⟨ Global definition selection 14 ⟩ ≡
  static LDGlobal *ld_lookup(ArchLD *ld, const char *name)
  {
    for (size_t i = 0; i < ld→global_count; ++i)
      if (¬strcmp(ld→globals[i].name, name)) return &ld→globals[i];
    return Λ;
  }
  static bool ld_collect_globals(ArchLD *ld)
  {
    for (size_t i = 0; i < ld→input_count; ++i) {
      ArchBFDObject * obj = ld→inputs[i].bfd;
      for (size_t j = 0; j < archbfd_object_symbol_count(obj); ++j) {
        const ArchBFDObjectSymbol*s = archbfd_object_symbol(obj, j);
        if (¬s ∨ ¬s→name ∨ ¬*s→name ∨ s→class_value ≡ LD_BIND_LOCAL ∨ s→shndx ≡
            LD_SHN_UNDEF ∨ (s→shndx ≥ 0 ∧ (size_t) s→shndx <
            archbfd_object_section_count(obj) ∧ ld→inputs[i].discard[s→shndx])) continue;
        if (s→class_value ≠ LD_BIND_GLOBAL ∧ s→class_value ≠ LD_BIND_WEAK ∧ s→class_value ≠ 10) return
            ld_fail(ld, "%s: unsupported symbol binding %" PRId64 , ld→inputs[i].path, s→class_value);
        int rank = s→shndx ≡ LD_SHN_COMMON ? 2 : (s→class_value ≡ LD_BIND_WEAK ? 1 : 3);
        if (s→shndx ≠ LD_SHN_COMMON ∧ s→shndx ≠ LD_SHN_ABS ∧ (s→shndx < 0 ∨ (size_t) s→shndx ≥
            archbfd_object_section_count(obj)))
          return ld_fail(ld, "%s: bad symbol section: %s", ld→inputs[i].path, s→name);
        LDGlobal *g = ld_lookup(ld, s→name);
        if (¬g) {
          if (¬ld_grow(ld, (void **) &ld→globals, &ld→global_cap, ld→global_count + 1, sizeof *ld→globals))
            return false;
          g = &ld→globals[ld→global_count++];
          memset(g, 0, sizeof *g);
          g→name = ld_strdup(s→name);
          if (¬g→name) return ld_fail(ld, "out of memory");
          g→input = i;
          g→symbol = j;
          g→rank = rank;
        }
        else if (rank ≡ 3 ∧ g→rank ≡ 3) {
          return ld_fail(ld, "duplicate strong definition: %s (%s, %s)", s→name,
              ld→inputs[g→input].path, ld→inputs[i].path);
        }
        else if (rank > g→rank) {
          g→input = i;
          g→symbol = j;
          g→rank = rank;
        }
        if (rank ≡ 2) {
          if (s→size < 0 ∨ s→value < 0) return ld_fail(ld, "bad COMMON symbol: %s", s→name);
          uint64_t a = s→value ? (uint64_t)s→value : 1;
          if (¬ld_power_two(a)) return ld_fail(ld, "bad COMMON alignment: %s", s→name);
          if (g→common_size < (uint64_t)s→size) g→common_size = (uint64_t)s→size;
```

```
            if (g→common_align < a) g→common_align = a;
          }
        }
      }
    }
    return true;
  }
  static bool ld_check_undefined(ArchLD *ld)
  {
    for (size_t i = 0; i < ld→input_count; ++i)
      for (size_t j = 0; j < archbfd_object_symbol_count(ld→inputs[i].bfd); ++j) {
        const ArchBFDObjectSymbol*s = archbfd_object_symbol(ld→inputs[i].bfd, j);
        if (s→class_value ≡ LD_BIND_GLOBAL ∧ s→shndx ≡ LD_SHN_UNDEF ∧ s→name ∧ *s→name ∧ ¬ld_lookup(ld,
              s→name) ∧ ¬(ld→elf32 ≡ false ∧ ¬strcmp(s→name, "_GLOBAL_OFFSET_TABLE_")))
          return ld_fail(ld, "%s: undefined reference to %s", ld→inputs[i].path, s→name);
      }
    return true;
  }
```

This code is used in section 3.

**15.**

**16. Placement and page boundaries.**
Text, read-only data, writable data, and zero-fill sections receive final virtual addresses.

```
⟨ Placement and page boundaries 16 ⟩ ≡
  static bool ld_place(ArchLD *ld) { uint64_t cursor = ld->base;
      for (unsigned cat = 0; cat < 4; ++cat) {
        if (cat ∧ cat ≤ 2 ∧ ¬ld_align(ld, cursor, 4096, &cursor)) return false;
        ld->cat_start[cat] = cursor − ld->base;
        for (size_t i = 0; i < ld->part_count; ++i) {
          LDPart *p = &ld->parts[i];
          if (p->category ≠ cat) continue;
          if (¬ld_align(ld, cursor, p->align, &cursor)) return false;
          if (cursor < ld->base ∨ p->length > UINT64_MAX − cursor)
            return ld_fail(ld, "image␣address␣overflow");
          p->address = cursor;
          p->image_offset = cursor − ld->base;
          cursor += p->length;
          if (cursor − ld->base > LD_MAX_IMAGE) return ld_fail(ld, "linked␣image␣exceeds␣512␣MiB");
        }
        if (cat ≡ 2) {
          if (¬ld_align(ld, cursor, 8, &cursor)) return false;
          ld->got_base = cursor;
          for (size_t i = 0; i < ld->got_count; i++) {
            if (cursor > UINT64_MAX − 8) return ld_fail(ld, "GOT␣address␣overflow");
            ld->got[i].address = cursor;
            ld->got[i].image_offset = cursor − ld->base;
            cursor += 8;
          }
        }
        ld->cat_end[cat] = cursor − ld->base;
      }
      ld->cat_start[3] = cursor − ld->base;
      for (size_t i = 0; i < ld->global_count; ++i) {
        LDGlobal *g = &ld->globals[i];
        if (g->rank ≠ 2) continue;
        if (¬ld_align(ld, cursor, g->common_align, &cursor)) return false;
        if (cursor < ld->base ∨ g->common_size > UINT64_MAX − cursor)
          return ld_fail(ld, "COMMON␣allocation␣overflow");
        g->common_addr = cursor;
        cursor += g->common_size;
        if (cursor − ld->base > LD_MAX_IMAGE) return ld_fail(ld, "linked␣image␣exceeds␣512␣MiB");
      }
      ld->cat_end[3] = cursor − ld->base;
      if (cursor < ld->base ∨ cursor − ld->base > SIZE_MAX)
        return ld_fail(ld, "linked␣image␣cannot␣fit␣memory");
      ld->image_size = (size_t)(cursor − ld->base); ld->image = ( uint8_t * )
          calloc(ld->image_size ? ld->image_size : 1, 1);
      if (¬ld->image) return ld_fail(ld, "out␣of␣memory␣for␣linked␣image");
      for (size_t i = 0; i < ld->part_count; ++i) {
        const LDPart *p = &ld->parts[i];
        const LDInput *in = &ld->inputs[p->input];
        const ArchBFDObjectSection*s = archbfd_object_section(in->bfd, p->section);
```

```
    if (s→type ≠ LD_SHT_NOBITS ∧ p→length)
      memcpy(ld→image + (size_t) p→image_offset, in→bytes + (size_t) s→offset, (size_t) p→length);
  }
  return true; }
```

This code is used in section 3.

**17.**

**18. Symbol address lookup.**
References are mapped to winning definitions, absolute symbols, and locally defined symbols.

```
⟨ Symbol address lookup 18 ⟩ ≡
  static bool ld_definition_address(ArchLD *ld, size_t input, size_t index, uint64_t * address)
  {
    if (input ≥ ld→input_count) return ld_fail(ld, "bad␣symbol␣input");
    const LDInput *in = &ld→inputs[input];
    const ArchBFDObjectSymbol*s = archbfd_object_symbol(in→bfd, index);
    if (¬s) return ld_fail(ld, "symbol␣index␣out␣of␣range");
    if (s→shndx ≡ LD_SHN_ABS) {
      if (s→value < 0) return ld_fail(ld, "negative␣absolute␣symbol");
      *address = (uint64_t)s→value;
      return true;
    }
    if (s→shndx < 0 ∨ (size_t) s→shndx ≥ archbfd_object_section_count(in→bfd))
      return ld_fail(ld, "invalid␣defining␣section␣of␣%s", s→name);
    size_t part = in→part_of_section[(size_t) s→shndx];
    if (part ≡ SIZE_MAX) return ld_fail(ld, "symbol␣%s␣defined␣outside␣allocated␣image", s→name);
    const LDPart *p = &ld→parts[part];
    if (s→value < 0 ∨ (uint64_t)s→value > p→length ∨ (uint64_t)s→value > UINT64_MAX − p→address)
      return ld_fail(ld, "symbol␣%s␣exceeds␣its␣section", s→name);
    *address = p→address + (uint64_t)s→value;
    return true;
  }
  static bool ld_global_address(ArchLD *ld, const LDGlobal *g, uint64_t * address)
  {
    if (g→rank ≡ 2) {
      *address = g→common_addr;
      return true;
    }
    return ld_definition_address(ld, g→input, g→symbol, address);
  }
  static bool ld_check_global_addresses(ArchLD *ld)
  {
    uint64_t address;
    for (size_t i = 0; i < ld→global_count; ++i)
      if (¬ld_global_address(ld, &ld→globals[i], &address)) return false;
    return true;
  }
  static bool ld_referenced_address(ArchLD *ld, size_t input, size_t index, uint64_t * address)
  {
    const ArchBFDObjectSymbol*s = archbfd_object_symbol(ld→inputs[input].bfd, index);
    if (¬s)
      return ld_fail(ld, "%s:␣relocation␣has␣bad␣symbol␣index␣%zu", ld→inputs[input].path, index);
    if (s→class_value ≡ LD_BIND_LOCAL) {
      if (s→shndx ≡ LD_SHN_UNDEF)
        return ld_fail(ld, "%s:␣unresolved␣local␣symbol", ld→inputs[input].path);
      return ld_definition_address(ld, input, index, address);
    }
```

```
    if (¬s→name ∨ ¬∗s→name)
      return ld_fail(ld, "%s: unnamed external relocation symbol", ld→inputs[input].path);
    if (¬ld→elf32 ∧ ¬strcmp(s→name, "_GLOBAL_OFFSET_TABLE_")) {
      ∗address = ld→got_base;
      return true;
    }
    LDGlobal ∗g = ld_lookup(ld, s→name);
    if (g) return ld_global_address(ld, g, address);
    if (s→class_value ≡ LD_BIND_WEAK) {
      ∗address = 0;
      return true;
    }
    return ld_fail(ld, "%s: undefined reference to %s", ld→inputs[input].path, s→name);
  }
```

This code is used in section 3.

**19.**

**20. Relocation validation.**
Only relocations with a supported lowering and consistent ArchBFD HOWTO are applied.

```
⟨ Relocation validation 20 ⟩ ≡
  typedef struct {
    unsigned bits;
    bool pcrel;
    bool signed_range;
    bool no_op;
  } LDHow;
  static bool ld_how(ArchLD *ld, int64_t number, LDHow *how)
  {
    memset(how, 0, sizeof *how);
    if (ld→elf32) {
      switch (number) {
      case 0: how→no_op = true;
        return true;
      case 1: how→bits = 32;
        return true;
      case 2: case 4: how→bits = 32;
        how→pcrel = true;
        how→signed_range = true;
        return true;
      default: return ld_fail(ld, "unsupported␣ELF32␣i386␣relocation␣%" PRId64 , number);
      }
    }
    switch (number) {
    case 0: how→no_op = true;
      return true;
    case 1: how→bits = 64;
      return true;
    case 2: case 4: case 9: case 41: case 42: how→bits = 32;
      how→pcrel = true;
      how→signed_range = true;
      return true;
    case 10: how→bits = 32;
      return true;
    case 11: how→bits = 32;
      how→signed_range = true;
      return true;
    case 12: how→bits = 16;
      return true;
    case 13: how→bits = 16;
      how→pcrel = true;
      how→signed_range = true;
      return true;
    case 14: how→bits = 8;
      return true;
    case 15: how→bits = 8;
      how→pcrel = true;
      how→signed_range = true;
      return true;
    case 24: how→bits = 64;
```

```
    how→pcrel = true;
    how→signed_range = true;
    return true;
  default: return ld_fail(ld,
      "relocation␣kind␣%" PRId64 "␣needs␣an␣architecture-specific␣lowering␣pass",
      number);
  }
}
static bool ld_validate_howto(ArchLD *ld, int64_t number, LDHow how)
{
  if (how.no_op) return true;
  int32_t k = archbfd_find_reloc_number(ld→spec, number);
  if (k < 0) return ld_fail(ld, "relocation␣%" PRId64 "␣absent␣from␣ArchBFD", number);
  const ArchBFDReloc *h = &ld→spec→relocations[k];
  if (h→size ≠ (int64_t)(how.bits/8) ∨ h→bitsize ≠ (int64_t)how.bits ∨ h→pcrel ≠ how.pcrel ∨ h→rightshift ≠
      0 ∨ h→bitpos ≠ 0) return ld_fail(ld, "ArchBFD␣HOWTO␣mismatch␣for␣%s", h→name);
  return true;
}
static uint64_t ld_load_le(const uint8_t *data, unsigned bytes)
{
  uint64_t x = 0;
  for (unsigned i = 0; i < bytes; ++i) x |= (uint64_t)data[i] ≪ (8 * i);
  return x;
}
static void ld_store_le(uint8_t * data, unsigned bytes, uint64_t v)
{
  for (unsigned i = 0; i < bytes; ++i) data[i] = (uint8_t)(v ≫ (8 * i));
}
static __int128 ld_sign_extend(uint64_t x, unsigned bits)
{
  if (bits ≡ 64) return (__int128)(int64_t)x;
  uint64_t top = (uint64_t)1 ≪ (bits − 1);
  x &= ((uint64_t)1 ≪ bits) − 1;
  return (__int128)(int64_t)((x ⊕ top) − top);
}
static bool ld_put_relocated(ArchLD *ld, uint8_t * data, LDHow how, __int128 value)
{
  unsigned bits = how.bits;
  __int128 min = how.signed_range ? −((__int128)1 ≪ (bits − 1)) : 0;
  __int128 max = how.signed_range ? (((__int128)1 ≪ (bits − 1)) − 1) : (((__int128)1 ≪ bits) − 1);
  if (bits ≡ 64 ∧ ¬how.signed_range) min = −((__int128)1 ≪ 63);
  if (value < min ∨ value > max) return ld_fail(ld, "relocation␣overflow:␣%u-bit␣field", bits);
  ld_store_le(data, bits/8, (uint64_t)value);
  return true;
}
```

This code is used in section 3.

**21.**

**22. Relocation section iteration.**
ArchBFD relocation records are matched against their ELF relocation-section targets.

```
⟨ Relocation section iteration 22 ⟩ ≡
  static bool ld_apply_one(ArchLD *, size_t, size_t, const ArchBFDObjectReloc*, bool);
  static bool ld_relocate(ArchLD *ld)
  {
    for (size_t i = 0; i < ld->input_count; ++i) {
      LDInput *in = &ld->inputs[i];
      size_t nsec = archbfd_object_section_count(in->bfd);
      size_t cursor = 0;
      size_t symtab = SIZE_MAX;
      for (size_t s = 0; s < nsec; ++s) {
        const ArchBFDObjectSection*sec = archbfd_object_section(in->bfd, s);
        if (sec->type ≡ LD_SHT_SYMTAB) {
          if (symtab ≠ SIZE_MAX)
            return ld_fail(ld, "%s: multiple symbol tables not supported", in->path);
          symtab = s;
        }
      }
      for (size_t rsec = 0; rsec < nsec; ++rsec) {
        const ArchBFDObjectSection*rs = archbfd_object_section(in->bfd, rsec);
        bool rela = rs->type ≡ LD_SHT_RELA;
        bool rel = rs->type ≡ LD_SHT_REL;
        if (¬rela ∧ ¬rel) continue;
        const ArchBFDRecord*record = archbfd_resolve_record(ld->spec, rela ? "rela" : "rel");
        if (¬record ∨ record->size ≤ 0 ∨ rs->size < 0 ∨ rs->offset < 0)
          return ld_fail(ld, "%s: invalid relocation section", in->path);
        int64_t entsize = rs->entsize ? rs->entsize : record->size;
        if (entsize < record->size ∨ rs->size % entsize ∨ ¬ld_span(in->bytes_len, rs->offset, rs->size))
          return ld_fail(ld, "%s: corrupt relocation entry extent", in->path);
        if (rs->info < 0 ∨ (size_t) rs->info ≥ nsec ∨ symtab ≡ SIZE_MAX ∨ rs->link < 0 ∨ (size_t) rs->link ≠
            symtab)
          return ld_fail(ld, "%s: unsupported relocation-to-symbol-table link", in->path);
        size_t target = (size_t) rs->info;
        size_t part = in->part_of_section[target];
        size_t count = (size_t)(rs->size / entsize);
        for (size_t k = 0; k < count; ++k) {
          if (cursor ≥ archbfd_object_reloc_count(in->bfd))
            return ld_fail(ld, "%s: truncated ArchBFD relocation stream", in->path);
          const ArchBFDObjectReloc*r = archbfd_object_reloc(in->bfd, cursor++);
          if (part ≡ SIZE_MAX) continue;
          if (¬ld_apply_one(ld, i, part, r, rela)) return false;
        }
      }
      if (cursor ≠ archbfd_object_reloc_count(in->bfd))
        return ld_fail(ld, "%s: unmatched ArchBFD relocations", in->path);
    }
    return true;
  }
```

This code is used in section 3.

**23.**

**24. Applying relocation fields.**
For PC-relative references calculate S + A - P; reject out-of-range values.

```
⟨ Applying relocation fields 24 ⟩ ≡
  static bool ld_apply_one(ArchLD *ld, size_t input, size_t part_index, const ArchBFDObjectReloc*r,
          bool explicit_addend)
  {
    LDInput *in = &ld→inputs[input];
    const LDPart *part = &ld→parts[part_index];
    const ArchBFDObjectSection*s = archbfd_object_section(in→bfd, part→section);
    LDHow how;
    if (¬ld_how(ld, r→type, &how)) return false;
    if (how.no_op) return true;
    if (¬ld_validate_howto(ld, r→type, how)) return false;
    uint64_t nbytes = how.bits/8;
    if (s→type ≡ LD_SHT_NOBITS ∨ r→offset < 0 ∨ (uint64_t)r→offset > part→length ∨ nbytes >
          part→length − (uint64_t)r→offset)
      return ld_fail(ld, "%s: relocation patch outside section %s", in→path, s→name ? s→name : "");
    if (r→symbol < 0 ∨ (uint64_t)r→symbol ≥ archbfd_object_symbol_count(in→bfd))
      return ld_fail(ld, "%s: relocation symbol index invalid", in→path);
    uint8_t * patch = ld→image + (size_t) part→image_offset + (size_t) r→offset;
    __int128 a;
    if (explicit_addend) a = (__int128)r→addend;
    else {
      uint64_t encoded = ld_load_le(patch, how.bits/8);
      a = (how.pcrel ∨ how.signed_range) ? ld_sign_extend(encoded, how.bits) : (__int128)encoded;
    }
    uint64_t sym = 0;
    if (r→type ≡ 9 ∨ r→type ≡ 41 ∨ r→type ≡ 42) {
      bool found = false;
      for (size_t k = 0; k < ld→got_count; k++)
        if (ld→got[k].input ≡ input ∧ ld→got[k].symbol ≡ (size_t) r→symbol) {
          sym = ld→got[k].address;
          found = true;
          break;
        }
      if (¬found) return ld_fail(ld, "missing generated GOT entry");
    }
    else if (¬ld_referenced_address(ld, input, (size_t) r→symbol, &sym)) return false;
    uint64_t place = part→address + (uint64_t)r→offset;
    __int128 value = (__int128)sym + a − (how.pcrel ? (__int128)place : 0);
    if (¬ld_put_relocated(ld, patch, how, value)) return ld_fail(ld, "%s: failed relocation", in→path);
    return true;
  }
```

This code is used in section 3.

**25.**

**26. Lazy archive extraction.**
GNU/BSD ar members are loaded on demand until no strong unresolved symbols can be satisfied.

⟨ Lazy archive extraction 26 ⟩ ≡ /* SysV, GNU and BSD ar, with lazy extraction driven by symbol demand. * Archive metadata never enters the actual object set until selected. */

```
static bool ld_archive_number(const char *s, size_t n, size_t *out)
{
  size_t v = 0;
  bool digit = false;
  for (size_t i = 0; i < n; i++) {
    if (s[i] ≡ '␣') {
      for (size_t k = i; k < n; k++)
        if (s[k] ≠ '␣') return false;
      break;
    }
    if (s[i] < '0' ∨ s[i] > '9' ∨ v > (SIZE_MAX − (size_t)(s[i] − '0'))/10) return false;
    v = v * 10 + (size_t)(s[i] − '0');
    digit = true;
  }
  if (¬digit) return false;
  *out = v;
  return true;
}
static bool ld_add_archive(ArchLD *ld, const char *path, uint8_t * bytes, size_t length)
{
  if (¬ld_grow(ld, (void **) &ld→archives, &ld→archive_cap, ld→archive_count + 1, sizeof(LDArchive)))
    {
    free(bytes);
    return false;
  }
  LDArchive *ar = &ld→archives[ld→archive_count++];
  memset(ar, 0, sizeof (*ar));
  ar→bytes = bytes;
  ar→length = length;
  ar→path = ld_strdup(path);
  if (¬ar→path) return ld_fail(ld, "out␣of␣memory");
  size_t pos = 8;
  const uint8_t*gnu_names = Λ;
  size_t names_size = 0;     /* GNU name table may precede members, as produced by ar. */
  while (pos < length) {
    if (length − pos < 60 ∨ memcmp(bytes + pos + 58, "'\n", 2))
      return ld_fail(ld, "%s:␣corrupt␣archive␣header␣at␣%zu", path, pos);
    const uint8_t*h = bytes + pos;
    size_t size = 0;
    if (¬ld_archive_number((const char *) h + 48, 10, &size))
      return ld_fail(ld, "%s:␣corrupt␣archive␣member␣size", path);
    if (size > length − pos − 60)
      return ld_fail(ld, "%s:␣archive␣member␣crosses␣file␣boundary", path);
    size_t off = pos + 60, dataoff = off, datasize = size;
    char namebuf[17];
    memcpy(namebuf, h, 16);
```

```
namebuf[16] = 0;
size_t name_len = 16;
while (name_len ∧ namebuf[name_len − 1] ≡ '␣') name_len−−;
namebuf[name_len] = 0;
char *name = Λ;
if (¬strcmp(namebuf, "//")) {
  gnu_names = bytes + off;
  names_size = size;
}
else if (¬strcmp(namebuf, "/") ∨ ¬strcmp(namebuf, "/SYM64/") ∨ ¬strncmp(namebuf,
    "__.SYMDEF", 9)) { }
else if (¬strncmp(namebuf, "#1/", 3)) {
  size_t n = 0;
  if (¬ld_archive_number(namebuf + 3, strlen(namebuf + 3), &n) ∨ n > size ∨ n > 4096)
    return ld_fail(ld, "%s:␣bad␣BSD␣extended␣name", path);
  name = malloc(n + 1);
  if (¬name) return ld_fail(ld, "out␣of␣memory");
  memcpy(name, bytes + off, n);
  name[n] = 0;
  dataoff += n;
  datasize −= n;
}
else if (namebuf[0] ≡ '/' ∧ namebuf[1] ≥ '0' ∧ namebuf[1] ≤ '9') {
  size_t n = 0;
  if (¬ld_archive_number(namebuf + 1, strlen(namebuf + 1), &n) ∨ ¬gnu_names ∨ n ≥ names_size)
    return ld_fail(ld, "%s:␣bad␣GNU␣name␣offset", path);
  const char *start = (const char *) gnu_names + n;
  size_t l = 0;
  while (n + l < names_size ∧ start[l] ≠ '/' ∧ start[l] ≠ '\n' ∧ start[l] ≠ 0) l++;
  name = malloc(l + 1);
  if (¬name) return ld_fail(ld, "out␣of␣memory");
  memcpy(name, start, l);
  name[l] = 0;
}
else {
  size_t n = name_len;
  if (n ∧ namebuf[n − 1] ≡ '/') n−−;
  name = malloc(n + 1);
  if (¬name) return ld_fail(ld, "out␣of␣memory");
  memcpy(name, namebuf, n);
  name[n] = 0;
}
if (name) {
  if (¬ld_grow(ld, (void **) &ar→members, &ar→cap, ar→count + 1, sizeof (*ar→members))) {
    free(name);
    return false;
  }
  LDMember *m = &ar→members[ar→count++];
  memset(m, 0, sizeof (*m));
```

```
      m→name = name;
      m→data_offset = dataoff;
      m→data_size = datasize;
    }
    pos = off + size + (size & 1);
    if (pos > length) return ld_fail(ld, "%s: truncated archive padding", path);
  }
  return true;
}
static bool ld_defined_now(ArchLD *ld, const char *name)
{
  for (size_t i = 0; i < ld→input_count; i++) {
    ArchBFDObject * obj = ld→inputs[i].bfd;
    for (size_t j = 0; j < archbfd_object_symbol_count(obj); j++) {
      const ArchBFDObjectSymbol*s = archbfd_object_symbol(obj, j);
      if (s→class_value ≠ LD_BIND_LOCAL ∧ s→shndx ≠ LD_SHN_UNDEF ∧ s→name ∧ ¬strcmp(name, s→name))
        return true;
    }
  }
  return false;
}
static bool ld_wanted_now(ArchLD *ld, const char *name)
{
  if (ld_defined_now(ld, name)) return false;
  for (size_t i = 0; i < ld→input_count; i++) {
    ArchBFDObject * obj = ld→inputs[i].bfd;
    for (size_t j = 0; j < archbfd_object_symbol_count(obj); j++) {
      const ArchBFDObjectSymbol*s = archbfd_object_symbol(obj, j);
      if (s→class_value ≡ LD_BIND_GLOBAL ∧ s→shndx ≡ LD_SHN_UNDEF ∧ s→name ∧ ¬strcmp(name, s→name))
        return true;
    }
  }
  return false;
}
static bool ld_archive_member_add(ArchLD *ld, LDArchive *ar, LDMember *m)
{
  if (¬ld_grow(ld, (void **) &ld→inputs, &ld→input_cap, ld→input_count + 1, sizeof (*ld→inputs)))
    return false;
  LDInput in = {0};
  size_t n = strlen(ar→path) + strlen(m→name) + 3;
  in.path = malloc(n);
  if (¬in.path) return ld_fail(ld, "out of memory");
  snprintf(in.path, n, "%s(%s)", ar→path, m→name);
  in.bytes = malloc(m→data_size ? m→data_size : 1);
  if (¬in.bytes) {
    free(in.path);
    return ld_fail(ld, "out of memory");
  }
  memcpy(in.bytes, ar→bytes + m→data_offset, m→data_size);
  in.bytes_len = m→data_size;
```

```
    DsymDiagsd;
    dsym_diags_init(&d);
    in.bfd = archbfd_object_open_memory(in.bytes, in.bytes_len, ld→spec, &d);
    if (¬in.bfd) ld_diagnostic(ld, in.path, &d);
    dsym_diags_free(&d);
    if (¬in.bfd ∨ strcmp(archbfd_object_kind(in.bfd),
            ld→elf32 ? "elf32" : "elf64") ∨ archbfd_object_machine(in.bfd) ≠ (ld→elf32 ? LD_EM_386 :
            LD_EM_X86_64) ∨ archbfd_object_type(in.bfd) ≠ LD_ET_REL) {
      if (in.bfd) ld_fail(ld, "%s: archive member has wrong ELF target", in.path);
      archbfd_object_close(in.bfd);
      free(in.bytes);
      free(in.path);
      return false;
    }
    size_t count = archbfd_object_section_count(in.bfd);
    in.part_of_section = malloc((count ? count : 1) * sizeof(size_t));
    in.discard = calloc(count ? count : 1, sizeof(bool));
    if (¬in.part_of_section ∨ ¬in.discard) {
      archbfd_object_close(in.bfd);
      free(in.bytes);
      free(in.path);
      free(in.part_of_section);
      free(in.discard);
      return ld_fail(ld, "out of memory");
    }
    for (size_t j = 0; j < count; j++) in.part_of_section[j] = SIZE_MAX;
    ld→inputs[ld→input_count++] = in;
    m→extracted = true;
    return true;
  }
  static bool ld_extract_archives(ArchLD *ld)
  {
    size_t loops = 0;
    bool progress;
    do {
      progress = false;
      for (size_t a = 0; a < ld→archive_count; a++) {
        LDArchive *ar = &ld→archives[a];
        for (size_t i = 0; i < ar→count; i++) {
          LDMember *m = &ar→members[i];
          if (m→extracted) continue;
          if (¬m→defs) {
            if (m→data_size < 4 ∨ memcmp(ar→bytes + m→data_offset, "\177ELF", 4)) continue;
            DsymDiagsd;
            dsym_diags_init(&d);
            ArchBFDObject * obj = archbfd_object_open_memory(ar→bytes + m→data_offset, m→data_size,
                ld→spec, &d);
            dsym_diags_free(&d);
            if (¬obj) continue;
            size_t cap = 0;
```

```
      for (size_t j = 0; j < archbfd_object_symbol_count(obj); j++) {
        const ArchBFDObjectSymbol *s = archbfd_object_symbol(obj, j);
        if (s->class_value ≡ LD_BIND_LOCAL ∨ s->shndx ≡ LD_SHN_UNDEF ∨ ¬s->name ∨ ¬*s->name)
          continue;
        if (¬ld_grow(ld, (void **) &m->defs, &cap, m->def_count + 1, sizeof(char *))) {
          archbfd_object_close(obj);
          return false;
        }
        m->defs[m->def_count] = ld_strdup(s->name);
        if (¬m->defs[m->def_count]) {
          archbfd_object_close(obj);
          return ld_fail(ld, "out of memory");
        }
        m->def_count++;
      }
      archbfd_object_close(obj);
      if (¬m->defs) {
        m->defs = calloc(1, sizeof(char *));
        if (¬m->defs) return ld_fail(ld, "out of memory");
      }
    }
    bool wanted = false;
    for (size_t z = 0; z < m->def_count; z++)
      if (ld_wanted_now(ld, m->defs[z])) {
        wanted = true;
        break;
      }
    if (wanted) {
      if (¬ld_archive_member_add(ld, ar, m)) return false;
      progress = true;
    }
  }
 }
 if (++loops > 100000) return ld_fail(ld, "archive extraction iteration limit exceeded");
} while (progress);
return true;
}
```

This code is used in section 3.

**27.**

**28. GOT synthesis.**
Produce initialized GOT cells for nonrelaxed GOTPCREL and GOTPCRELX instructions.

```
⟨ GOT synthesis 28 ⟩ ≡      /* GOT entries are real data cells. The non-relaxing GOTPCRELX lowering
      keeps * the original machine instruction and directs it to a populated GOT slot. */
  static bool ld_collect_got(ArchLD *ld) {
    if (ld→elf32) return true;
    for (size_t i = 0; i < ld→input_count; i++) { LDInput *in = &ld→inputs[i];
    size_t nsec = archbfd_object_section_count(in→bfd);
    size_t cursor = 0; for (size_t j = 0; j < nsec; j++) { const
        ArchBFDObjectSection*sec = archbfd_object_section(in→bfd, j);
    if (sec→type ≠ LD_SHT_RELA ∧ sec→type ≠ LD_SHT_REL) continue;
    const ArchBFDRecord*rec = archbfd_resolve_record(ld→spec,
        sec→type ≡ LD_SHT_RELA ? "rela" : "rel");
    int64_t entsize = sec→entsize > 0 ? sec→entsize : rec ? rec→size : 0;
    size_t cnt = entsize > 0 ∧ sec→size ≥ 0 ? (size_t)(sec→size / entsize) : 0;
    bool active = sec→info ≥ 0 ∧ (size_t) sec→info < nsec ∧ in→part_of_section[sec→info] ≠ SIZE_MAX;
        for (size_t k = 0; k < cnt; k++) {
    if (cursor ≥ archbfd_object_reloc_count(in→bfd))
      return ld_fail(ld, "ArchBFD relocation stream inconsistent");
    const ArchBFDObjectReloc*r = archbfd_object_reloc(in→bfd, cursor++);
    if (¬active ∨ (r→type ≠ 9 ∧ r→type ≠ 41 ∧ r→type ≠ 42)) continue;
    if (r→symbol < 0 ∨ (size_t) r→symbol ≥ archbfd_object_symbol_count(in→bfd))
      return ld_fail(ld, "invalid GOT relocation symbol");
    bool exists = false;
    for (size_t z = 0; z < ld→got_count; z++)
      if (ld→got[z].input ≡ i ∧ ld→got[z].symbol ≡ (size_t) r→symbol) {
        exists = true;
        break;
      }
    if (exists) continue;
    if (¬ld_grow(ld, (void **) &ld→got, &ld→got_cap, ld→got_count + 1, sizeof (*ld→got))) return false;
    ld→got[ld→got_count++] = (LDGOT) { . input = i , . symbol = (size_t) r→symbol } ; } } } return
          true; } static bool ld_fill_got(ArchLD *ld)
    {
      for (size_t i = 0; i < ld→got_count; i++) {
        LDGOT *e = &ld→got[i];
        uint64_t v = 0;
        if (¬ld_referenced_address(ld, e→input, e→symbol, &v)) return false;
        if (e→image_offset > ld→image_size ∨ ld→image_size − e→image_offset < 8)
          return ld_fail(ld, "GOT entry out of bounds");
        ld_store_le(ld→image + (size_t) e→image_offset, 8, v);
      }
      return true;
    }
```

This code is used in section 3.

**29.**

**30. Link phases and diagnostics.**
The phases are executed deterministically and expose a symbol and map inspection API.

```
⟨ Link phases and diagnostics 30 ⟩ ≡
  bool archld_link(ArchLD *ld) { if ( ¬ld ∨ ld→error [0] ∨ ld→linked ) return false;
      if (¬ld→input_count) return ld_fail(ld, "no input object files");
      if (¬ld_extract_archives(ld) ∨ ¬ld_prepare_groups(ld) ∨ ¬ld_collect_parts(ld) ∨ ¬ld_collect_globals(ld) ∨
            ¬ld_collect_got(ld) ∨ ¬ld_check_undefined(ld) ∨ ¬ld_place(ld) ∨ ¬ld_check_global_addresses(ld) ∨
            ¬ld_fill_got(ld) ∨ ¬ld_relocate(ld)) return false;
      ld→linked = true;
      return true; } const uint8_t*archld_image(const ArchLD *ld)
      {
        return ld ∧ ld→linked ? ld→image : Λ;
      }
      size_t archld_image_size(const ArchLD *ld)
      {
        return ld ∧ ld→linked ? ld→image_size : 0;
      }
      uint64_t archld_image_base(const ArchLD *ld)
      {
        return ld ? ld→base : 0;
      }
      bool archld_symbol(const ArchLD *ld, const char *name, uint64_t * address)
      {
        if (¬ld ∨ ¬ld→linked ∨ ¬name ∨ ¬address) return false;
        for (size_t i = 0; i < ld→global_count; ++i) {
          const LDGlobal *g = &ld→globals[i];
          if (strcmp(g→name, name)) continue;
          if (g→rank ≡ 2) {
            *address = g→common_addr;
            return true;
          }
          return ld_definition_address((ArchLD *) ld, g→input, g→symbol, address);
        }
        return false;
      }
      void archld_map(const ArchLD *ld, FILE *out)
      {
        if (¬ld ∨ ¬ld→linked ∨ ¬out) return;
        fprintf(out, "ArchLD link map: %s; base=0x%016" PRIx64 "; image=%zu bytes\n",
            ld→elf32 ? "ELF32 i386" : "ELF64 x86-64", ld→base, ld→image_size);
        fprintf(out, "Sections (virtual addresses):\n");
        for (unsigned category = 0; category < 4; ++category)
          for (size_t i = 0; i < ld→part_count; ++i) {
            const LDPart *p = &ld→parts[i];
            if (p→category ≠ category) continue;
            const LDInput *in = &ld→inputs[p→input];
            const ArchBFDObjectSection*s = archbfd_object_section(in→bfd, p→section);
            fprintf(out, "  %016" PRIx64 "  %8" PRIu64 "  %-18s %s\n", p→address, p→length,
                s→name ? s→name : "", in→path);
          }
```

```
    fprintf(out, "Global definitions:\n");
    for (size_t i = 0; i < ld->global_count; ++i) {
      const LDGlobal *g = &ld->globals[i];
      uint64_t value = 0;
      if (ld_global_address((ArchLD *) ld, g, &value))
        fprintf(out, "  %016" PRIx64 "  %-30s %s%s\n", value, g->name, ld->inputs[g->input].path,
            g->rank == 2 ? " (COMMON)" : "");
    }
  }
```

This code is used in section 3.

**31.**

**32. Executable ELF64 output.**
Page-congruent W-X load segments, zero fill, deterministic symbol and string tables.

⟨ Executable ELF64 output 32 ⟩ ≡ /* Production output layer: a real, page-congruent, non-PIE static ELF64 * executable, with W-X PT_LOADs and useful section/symbol tables. * All source input section and relocation metadata comes from ArchBFD. */

```
#include <elf.h>
#include <sys/stat.h>
#include <unistd.h>
#include <fcntl.h>
#if __BYTE_ORDER__ ≠ __ORDER_LITTLE_ENDIAN__
#error "ArchLD standalone ELF64 writer needs a little-endian build host"
#endif
  size_t archld_global_count(const ArchLD *ld)
  {
    return ld ∧ ld→linked ? ld→global_count : 0;
  }
  bool archld_global_at(const ArchLD *ld, size_t i, const char **name, uint64_t * addr, uint64_t * sz)
  {
    if (¬ld ∨ ¬ld→linked ∨ i ≥ ld→global_count) return false;
    const LDGlobal *g = &ld→globals[i];
    if (name) *name = g→name;
    if (addr ∧ ¬ld_global_address((ArchLD *) ld, g, addr)) return false;
    if (sz) {
      const ArchBFDObjectSymbol*s = archbfd_object_symbol(ld→inputs[g→input].bfd, g→symbol);
      *sz = g→rank ≡ 2 ? g→common_size : s ∧ s→size ≥ 0 ? (uint64_t)s→size : 0;
    }
    return true;
  }
  bool archld_write_map_file(const ArchLD *ld, const char *path)
  {
    FILE *f = fopen(path, "w");
    if (¬f) return false;
    archld_map(ld, f);
    bool ok = ¬ferror(f) ∧ fclose(f) ≡ 0;
    if (¬ok) remove(path);
    return ok;
  }
  static bool ld_buf_append(DsymBuf * b, const char *p, size_t n)
  {
    if (n > SIZE_MAX − b→len − 1) return false;
    if (b→len + n + 1 > b→cap) {
      size_t cap = b→cap ? b→cap : 64;
      while (cap < b→len + n + 1) {
        if (cap > SIZE_MAX/2) return false;
        cap *= 2;
      }
      char *q = realloc(b→data, cap);
      if (¬q) return false;
      b→data = q;
```

```
    b→cap = cap;
  }
  memcpy(b→data + b→len, p, n);
  b→len += n;
  b→data[b→len] = 0;
  return true;
}
static bool ld_write_all(FILE *f, const void *data, size_t n)
{
  return fwrite(data, 1, n, f) ≡ n;
}
static bool ld_seek(FILE *f, uint64_t offset)
{
  if (offset > INT64_MAX) return false;
  return fseeko(f, (off_t) offset, SEEK_SET) ≡ 0;
}
static uint64_t ld_align_up(uint64_t n, uint64_t a)
{
  return (n + a − 1) & ∼(a − 1);
}
static uint32_t ld_symbol_section(const ArchLD *ld, const LDGlobal *g, uint64_t address)
{
  const ArchBFDObjectSymbol *s = archbfd_object_symbol(ld→inputs[g→input].bfd, g→symbol);
  if (s→shndx ≡ LD_SHN_ABS ∧ g→rank ≠ 2) return SHN_ABS;
  if (g→rank ≡ 2) return 4;
  for (unsigned i = 0; i < 4; i++) {
    if (address ≥ ld→base + ld→cat_start[i] ∧ address ≤ ld→base + ld→cat_end[i]) return (uint32_t)(i + 1);
  }
  return SHN_UNDEF;
}   /* Create the output alongside its destination; never truncate the original * file before all
       relocations, metadata and writes have succeeded. */
static FILE *ld_open_atomic(ArchLD *ld, const char *path, char **temp)
{
  size_t len = strlen(path);
  if (len > SIZE_MAX − 24) {
    ld_fail(ld, "output filename too long");
    return Λ;
  }
  char *name = malloc(len + 20);
  if (¬name) {
    ld_fail(ld, "out of memory");
    return Λ;
  }
  sprintf(name, "%s.archld-XXXXXX", path);
  int fd = mkstemp(name);
  if (fd < 0) {
    ld_fail(ld, "cannot create output next to %s: %s", path, strerror(errno));
    free(name);
    return Λ;
```

```
    }
    FILE *f = fdopen(fd, "wb");
    if (¬f) {
      ld_fail(ld, "cannot␣open␣temporary␣output:␣%s", strerror(errno));
      close(fd);
      unlink(name);
      free(name);
      return Λ;
    }
    *temp = name;
    return f;
  }
  static bool ld_close_atomic(ArchLD *ld, FILE *f, char *temp, const char *path, bool ok)
  {
    if (ok ∧ fflush(f)) ok = false;
    if (ok ∧ fsync(fileno(f))) ok = false;
    if (fclose(f)) ok = false;
    if (ok ∧ chmod(temp, °755)) ok = false;
    if (ok ∧ rename(temp, path)) ok = false;
    if (¬ok) {
      ld_fail(ld, "cannot␣publish␣executable␣%s:␣%s", path, strerror(errno));
      unlink(temp);
    }
    free(temp);
    return ok;
  }
```

This code is used in section 3.

**33.**

**34. Executable ELF32 output.**
Parallel ELFCLASS32 writer with genuine 32-bit program, section and symbol entries.

```
⟨ Executable ELF32 output 34 ⟩ ≡
  static bool ld_write_elf32(ArchLD *, const char *, uint64_t); bool archld_write_elf(ArchLD
        *ld, const char *path, uint64_t entry) {
    if (ld ∧ ld→elf32) return ld_write_elf32(ld, path, entry);
    if (¬ld ∨ ¬ld→linked ∨ ¬path ∨ ¬path[0]) return false;
    if (ld→base < 4096 ∨ ld→base % 4096 ∨ entry < ld→base + ld→cat_start[0] ∨ entry ≥ ld→base + ld→cat_end[0])
      return ld_fail(ld, "ELF64 executable requires page-aligned base >= 0x1000 and e\
          ntry inside executable text");
    if (ld→image_size > LD_MAX_IMAGE ∨ ld→global_count > UINT32_MAX)
      return ld_fail(ld, "ELF output size exceeds limits");
    enum {
      SEC_COUNT = 8, SEG_COUNT = 5
    };
    const char *labels[SEC_COUNT] = {"", ".text", ".rodata", ".data", ".bss", ".symtab",
        ".strtab", ".shstrtab"};
    uint32_t shnames[SEC_COUNT] = {0};
    DsymBuf shstr, str;
    dsym_buf_init(&shstr);
    dsym_buf_init(&str);
    bool ok = dsym_buf_puts(&shstr, "") ∧ dsym_buf_puts(&str, "");
      /* DsymBuf puts() appends zero bytes only for C strings; write initial NUL explicitly. */
    char zero = 0;
    if (ok) {
      ok = ld_buf_append(&shstr, &zero, 1) ∧ ld_buf_append(&str, &zero, 1);
    }
    for (size_t i = 1; i < SEC_COUNT ∧ ok; i++) {
      shnames[i] = (uint32_t)shstr.len;
      ok = ld_buf_append(&shstr, labels[i], strlen(labels[i]) + 1);
    }
    Elf64_Sym * symbols = Λ;
    uint32_t * nameoff = Λ;
    size_t symcount = ld→global_count + 1;
    if (ok) {
      symbols = calloc(symcount, sizeof (*symbols));
      nameoff = calloc(symcount, sizeof (*nameoff));
      ok = symbols ∧ nameoff;
    }
    for (size_t i = 0; i < ld→global_count ∧ ok; i++) {
      const LDGlobal *g = &ld→globals[i];
      uint64_t a = 0, sz = 0;
      ok = archld_global_at(ld, i, Λ, &a, &sz);
      if (¬ok) break;
      if (str.len > UINT32_MAX) {
        ok = false;
        break;
      }
      nameoff[i + 1] = (uint32_t)str.len;
      ok = ld_buf_append(&str, g→name, strlen(g→name) + 1);
```

```
  symbols[i + 1].st_name = nameoff[i + 1];
  symbols[i + 1].st_info = ELF64_ST_INFO(g→rank ≡ 1 ? STB_WEAK : STB_GLOBAL, STT_NOTYPE);
  symbols[i + 1].st_shndx = (Elf64_Half) ld_symbol_section(ld, g, a);
  symbols[i + 1].st_value = a;
  symbols[i + 1].st_size = sz;
}
if (¬ok) {
  free(symbols);
  free(nameoff);
  dsym_buf_free(&str);
  dsym_buf_free(&shstr);
  return ld_fail(ld, "out␣of␣memory␣building␣ELF␣symbol␣table");
}
uint64_t max_file = ld→cat_end[2];
if (ld→cat_end[1] > max_file) max_file = ld→cat_end[1];
if (ld→cat_end[0] > max_file) max_file = ld→cat_end[0];
uint64_t syoff = ld_align_up(4096 + max_file, 8);
uint64_t stroff = syoff + symcount * sizeof (Elf64_Sym);
uint64_t shstroff = stroff + str.len;
uint64_t shoff = ld_align_up(shstroff + shstr.len, 8);
if (shoff > UINT64_MAX − SEC_COUNT * sizeof (Elf64_Shdr) ∨ shoff > INT64_MAX) {
  ok = false;
  goto done;
}
Elf64_Ehdr eh = {0};
memcpy(eh.e_ident, ELFMAG, SELFMAG);
eh.e_ident[EI_CLASS] = ELFCLASS64;
eh.e_ident[EI_DATA] = ELFDATA2LSB;
eh.e_ident[EI_VERSION] = EV_CURRENT;
eh.e_ident[EI_OSABI] = ELFOSABI_SYSV;
eh.e_type = ET_EXEC;
eh.e_machine = EM_X86_64;
eh.e_version = EV_CURRENT;
eh.e_entry = entry;
eh.e_phoff = sizeof (eh);
eh.e_ehsize = sizeof (eh);
eh.e_phentsize = sizeof (Elf64_Phdr);
eh.e_phnum = SEG_COUNT;
eh.e_shoff = shoff;
eh.e_shentsize = sizeof (Elf64_Shdr);
eh.e_shnum = SEC_COUNT;
eh.e_shstrndx = 7;
Elf64_Phdr ph[SEG_COUNT] = {0}; ph[0] = (Elf64_Phdr) { . p_type = PT_LOAD , . p_flags = PF_R
    , . p_offset = 0 , . p_vaddr = ld→base − 4096 , . p_paddr = ld→base − 4096 , .
    p_filesz = 4096 , . p_memsz = 4096 , . p_align = 4096 } ; for (int i = 0; i < 3; i++) {
    uint64_t start = ld→cat_start[i], end = ld→cat_end[i]; ph[i + 1] = (Elf64_Phdr) { .
    p_type = PT_LOAD , . p_flags = (i ≡ 0 ? PF_R | PF_X : i ≡ 1 ? PF_R : PF_R | PF_W) , .
    p_offset = 4096 + start , . p_vaddr = ld→base + start , . p_paddr = ld→base + start , .
    p_filesz = end − start , . p_memsz = i ≡ 2 ? ld→image_size − start : end − start , .
    p_align = 4096 } ; }
  for (int i = 1; i ≤ 3; i++)
    if (ph[i].p_memsz ≡ 0) memset(&ph[i], 0, sizeof (ph[i]));
```

```
    ph[4] = (Elf64_Phdr) { . p_type = PT_GNU_STACK , . p_flags = PF_R | PF_W , . p_align = 16 }
        ;
    Elf64_Shdr sh[SEC_COUNT] = {0}; for (int i = 1; i ≤ 4; i++) {
            uint64_t st = ld→cat_start[i − 1], end = ld→cat_end[i − 1];
            sh[i] = (Elf64_Shdr) { . sh_name = shnames[i] , .
            sh_type = i ≡ 4 ? SHT_NOBITS : SHT_PROGBITS , . sh_flags =
            SHF_ALLOC | (i ≡ 1 ? SHF_EXECINSTR : i ≥ 3 ? SHF_WRITE : 0)
            , . sh_addr = ld→base + st , . sh_offset = 4096 + st ,
            . sh_size = i ≡ 4 ? ld→image_size − st : end − st , .
            sh_addralign = i ≡ 4 ? 16 : 4096 } ; } sh[5] = (Elf64_Shdr)
            { . sh_name = shnames[5] , . sh_type = SHT_SYMTAB , .
            sh_offset = syoff , . sh_size = symcount ∗ sizeof (Elf64_Sym)
            , . sh_link = 6 , . sh_info = 1 , . sh_addralign = 8 , .
            sh_entsize = sizeof (Elf64_Sym) } ; sh[6] = (Elf64_Shdr)
            { . sh_name = shnames[6] , . sh_type = SHT_STRTAB , .
            sh_offset = stroff , . sh_size = str.len , . sh_addralign = 1
            } ; sh[7] = (Elf64_Shdr) { . sh_name = shnames[7]
            , . sh_type = SHT_STRTAB , . sh_offset = shstroff , .
            sh_size = shstr.len , . sh_addralign = 1 } ;
        char ∗tmp = Λ;
        FILE ∗f = ld_open_atomic(ld, path, &tmp);
        if (¬f) {
          ok = false;
          goto done;
        }
        ok = ld_write_all(f, &eh, sizeof (eh)) ∧ ld_write_all(f, ph, sizeof
            (ph));
        if (ok) {
            /∗ Do not materialize SHT_NOBITS/common into the file. ∗/
          for (unsigned i = 0; i < 3 ∧ ok; i++) {
            uint64_t a = ld→cat_start[i], b = ld→cat_end[i];
            if (a ≡ b) continue;
            ok = ld_seek(f, 4096 + a) ∧ ld_write_all(f, ld→image + (size_t) a,
                (size_t)(b − a));
          }
          ok = ok ∧ ld_seek(f, syoff) ∧ ld_write_all(f, symbols,
              symcount ∗ sizeof (∗symbols)) ∧ ld_write_all(f, str.data,
              str.len) ∧ ld_write_all(f, shstr.data, shstr.len) ∧ ld_seek(f,
              shoff) ∧ ld_write_all(f, sh, sizeof (sh));
        }
        ok = ld_close_atomic(ld, f, tmp, path, ok);
      done: free(symbols);
        free(nameoff);
        dsym_buf_free(&shstr);
        dsym_buf_free(&str);
        return ok; }
          /∗ ELFCLASS32 i386 output shares section placement, symbol
            selection and the * ArchBFD-driven input machinery with the
            ELF64 backend. The output ABI is * genuinely 32-bit: headers,
            program headers, section headers and symbols. ∗/
        static bool ld_write_elf32(ArchLD ∗ld, const char
              ∗path, uint64_t entry) {
```

```
if (¬ld ∨ ¬ld→linked ∨ ld→base < 4096 ∨ ld→base % 4096 ∨ entry >
      UINT32_MAX ∨ ld→base + ld→image_size >
      UINT32_MAX ∨ entry < ld→base + ld→cat_start[0] ∨ entry ≥
      ld→base + ld→cat_end[0]) return ld_fail(ld,
      "ELF32␣requires␣a␣page-aligned␣32-bit␣ad\
      dress␣range␣and␣an␣executable␣entry");
enum {
  SEC_COUNT = 8, SEG_COUNT = 5
};
const char *labels[SEC_COUNT] = {"", ".text", ".rodata",
    ".data", ".bss", ".symtab", ".strtab", ".shstrtab"};
uint32_t shnames[SEC_COUNT] = {0};
DsymBuf shstr, str;
dsym_buf_init(&shstr);
dsym_buf_init(&str);
char zero = 0;
bool ok = ld_buf_append(&shstr, &zero,
    1) ∧ ld_buf_append(&str, &zero, 1);
for (int i = 1; i < SEC_COUNT ∧ ok; i++) {
  shnames[i] = (uint32_t)shstr.len;
  ok = ld_buf_append(&shstr, labels[i], strlen(labels[i]) + 1);
}
size_t n = ld→global_count + 1;
Elf32_Sym * symbols = calloc(n, sizeof (Elf32_Sym));
if (¬symbols) ok = false;
for (size_t i = 0; i < ld→global_count ∧ ok; i++) { const
    LDGlobal *g = &ld→globals[i];
uint64_t addr = 0, sz = 0;
if (¬archld_global_at(ld, i, Λ, &addr, &sz) ∨ addr >
      UINT32_MAX ∨ sz > UINT32_MAX ∨ str.len > UINT32_MAX)
  {
  ok = false;
  break;
}
symbols[i + 1] = (Elf32_Sym) { . st_name = (uint32_t)str.len , .
        st_value = (uint32_t)addr , . st_size = (uint32_t)sz
        , . st_info = ELF32_ST_INFO(g→rank ≡ 1 ?
        STB_WEAK : STB_GLOBAL, STT_NOTYPE) , .
        st_shndx = (uint16_t)ld_symbol_section(ld, g, addr) } ;
    ok = ld_buf_append(&str, g→name, strlen(g→name) + 1); }
    if (¬ok) {
      ld_fail(ld,
          "invalid␣ELF32␣symbols␣or␣out␣of␣memory");
      goto done;
    }
    uint64_t max_file = ld→cat_end[2];
    if (ld→cat_end[0] > max_file) max_file = ld→cat_end[0];
    if (ld→cat_end[1] > max_file) max_file = ld→cat_end[1];
    uint64_t syoff = ld_align_up(4096 + max_file, 4);
    uint64_t stroff = syoff + n * sizeof (Elf32_Sym),
        shstroff = stroff + str.len;
```

```
uint64_t shoff = ld_align_up(shstroff + shstr.len, 4);
if (shoff + SEC_COUNT * sizeof (Elf32_Shdr) > UINT32_MAX)
  {
  ok = ld_fail(ld, "ELF32 output too large");
  goto done;
}
Elf32_Ehdr eh = {0};
memcpy(eh.e_ident, ELFMAG, SELFMAG);
eh.e_ident[EI_CLASS] = ELFCLASS32;
eh.e_ident[EI_DATA] = ELFDATA2LSB;
eh.e_ident[EI_VERSION] = EV_CURRENT;
eh.e_ident[EI_OSABI] = ELFOSABI_SYSV;
eh.e_type = ET_EXEC;
eh.e_machine = EM_386;
eh.e_version = EV_CURRENT;
eh.e_entry = (uint32_t)entry;
eh.e_phoff = sizeof (eh);
eh.e_ehsize = sizeof (eh);
eh.e_phentsize = sizeof (Elf32_Phdr);
eh.e_phnum = SEG_COUNT;
eh.e_shoff = (uint32_t)shoff;
eh.e_shentsize = sizeof (Elf32_Shdr);
eh.e_shnum = SEC_COUNT;
eh.e_shstrndx = 7;
Elf32_Phdr ph[SEG_COUNT] = {0}; ph[0] = (Elf32_Phdr)
        { . p_type = PT_LOAD , .
        p_flags = PF_R , . p_offset = 0 , .
        p_vaddr = (uint32_t)(ld->base - 4096) , .
        p_paddr = (uint32_t)(ld->base - 4096) , .
        p_filesz = 4096 , . p_memsz = 4096 , .
        p_align = 4096 } ; for (int i = 0; i < 3; i++)
        { uint64_t st = ld->cat_start[i],
        end = ld->cat_end[i]; ph[i + 1] = (Elf32_Phdr)
        { . p_type = PT_LOAD , . p_flags = i ≡ 0 ?
        PF_R | PF_X : i ≡ 1 ? PF_R : PF_R | PF_W
        , . p_offset = (uint32_t)(4096 + st) , .
        p_vaddr = (uint32_t)(ld->base + st) , .
        p_paddr = (uint32_t)(ld->base + st) , .
        p_filesz = (uint32_t)(end - st) , . p_memsz =
        (uint32_t)(i ≡ 2 ? ld->image_size - st : end - st)
        , . p_align = 4096 } ; }
      for (int i = 1; i ≤ 3; i++)
        if (ph[i].p_memsz ≡ 0)
          memset(&ph[i], 0, sizeof (ph[i]));
      ph[4] = (Elf32_Phdr) { . p_type = PT_GNU_STACK ,
            . p_flags = PF_R | PF_W , . p_align = 16 } ;
          Elf32_Shdr sh[SEC_COUNT] = {0}; for (int
                  i = 1; i ≤ 4; i++)
                  { uint64_t st =
                  ld->cat_start[i - 1],
                  end = ld->cat_end[i - 1];
                  sh[i] = (Elf32_Shdr) { .
```

```
        sh_name = shnames[i]
        , . sh_type = i ≡
        4 ? SHT_NOBITS :
        SHT_PROGBITS
        , . sh_flags =
        SHF_ALLOC | (i ≡ 1 ?
        SHF_EXECINSTR : i ≥
        3 ? SHF_WRITE : 0)
        , . sh_addr =
        (uint32_t)(ld→base + st)
        , . sh_offset =
        (uint32_t)(4096 + st) , .
        sh_size = (uint32_t)(i ≡
        4 ? ld→image_size − st :
        end − st) , .
        sh_addralign = i ≡
        4 ? 16 : 4096 } ; }
        sh[5] = (Elf32_Shdr) { .
        sh_name = shnames[5] ,
        . sh_type = SHT_SYMTAB
        , . sh_offset =
        (uint32_t)syoff
        , . sh_size =
        (uint32_t)(n ∗ sizeof
        (Elf32_Sym)) ,
        . sh_link = 6 , .
        sh_info = 1 , .
        sh_addralign = 4 , .
        sh_entsize = sizeof
        (Elf32_Sym) } ;
        sh[6] = (Elf32_Shdr) { .
        sh_name = shnames[6] ,
        . sh_type = SHT_STRTAB
        , . sh_offset =
        (uint32_t)stroff
        , . sh_size =
        (uint32_t)str.len , .
        sh_addralign = 1 } ;
        sh[7] = (Elf32_Shdr) { .
        sh_name = shnames[7] ,
        . sh_type = SHT_STRTAB
        , . sh_offset =
        (uint32_t)shstroff
        , . sh_size =
        (uint32_t)shstr.len , .
        sh_addralign = 1 } ;
    char ∗tmp = Λ;
    FILE
        ∗f = ld_open_atomic(ld,
        path, &tmp);
    if (¬f) {
      ok = false;
```

```
    goto done;
  }
  ok = ld_write_all(f,
      &eh, sizeof
      (eh)) ∧ ld_write_all(f,
      ph, sizeof (ph));
  for (int i = 0; i < 3 ∧ ok;
      i++) {
    uint64_t st =
        ld→cat_start[i],
        end = ld→cat_end[i];
    if (st ≠ end)
      ok = ld_seek(f, 4096 +
          st) ∧ ld_write_all(f,
          ld→image +
          (size_t) st,
          (size_t)(end − st));
  }
  ok = ok ∧ ld_seek(f,
      syoff) ∧ ld_write_all(f,
      symbols, n ∗ sizeof
      (∗symbols)) ∧
      ld_write_all(f, str.data,
      str.len) ∧ ld_write_all(f,
      shstr.data,
      shstr.len) ∧ ld_seek(f,
      shoff) ∧ ld_write_all(f,
      sh, sizeof (sh));
  ok = ld_close_atomic(ld, f,
      tmp, path, ok);
done: free(symbols);
  dsym_buf_free(&shstr);
  dsym_buf_free(&str);
  return ok; }
```

This code is used in section 3.

**35.**


__BYTE_ORDER__: 32.
__cplusplus: 2.
__int128: 20, 24.
__ORDER_LITTLE_ENDIAN__: 32.
a: 10, 26.
active: 28.
addend: 24.
addr: 32, 34.
address: 2, 6, 16, 18, 24, 30, 32.
align: 6, 12, 16.
ar: 10, 26.
arch: 10.
ArchBFD: 6.
ARCHBFD_BO_LITTLE: 10.
archbfd_close: 10.
archbfd_find_reloc_number: 20.
ARCHBFD_FMT_ELF: 10.
archbfd_load_name: 10.
archbfd_object_close: 10, 26.
archbfd_object_kind: 10, 26.
archbfd_object_machine: 10, 26.
archbfd_object_open_memory: 10, 26.
archbfd_object_reloc: 22, 28.
archbfd_object_reloc_count: 22, 28.
archbfd_object_section: 12, 16, 22, 24, 28, 30.
archbfd_object_section_count: 10, 12, 14, 18, 22, 26, 28.
archbfd_object_symbol: 12, 14, 18, 26, 32.
archbfd_object_symbol_count: 12, 14, 24, 26, 28.
archbfd_object_type: 10, 26.

*archbfd_open*: 10.
*archbfd_resolve_record*: 22, 28.
*ArchBFDObject*: 6, 14, 26.
*ArchBFDObjectReloc*: 22, 24, 28.
*ArchBFDObjectSection*: 12, 16, 22, 24, 28, 30.
*ArchBFDObjectSymbol*: 12, 14, 18, 26, 32.
*ArchBFDRecord*: 22, 28.
*ArchBFDReloc*: 20.
*ArchBFDSpec*: 6.
*archive_cap*: 6, 26.
*archive_count*: 6, 10, 26.
*archives*: 6, 10, 26.
**ArchLD**: 2, 6, 8, 10, 12, 14, 16, 18, 20, 22, 24, 26, 28, 30, 32, 34.
*archld_add*: 2, 10.
*archld_error*: 2, 10.
*archld_free*: 2, 10.
*archld_global_at*: 2, 32, 34.
*archld_global_count*: 2, 32.
*archld_image*: 2, 30.
*archld_image_base*: 2, 30.
*archld_image_size*: 2, 30.
*archld_link*: 2, 30.
*archld_map*: 2, 30, 32.
*archld_new*: 2, 10.
*archld_symbol*: 2, 30.
*archld_write_elf*: 2, 34.
*archld_write_map_file*: 2, 32.
`ARCHTOOLCHAIN_ARCHLD_H`: 2.
*args*: 8.
*bad*: 10.
*base*: 2, 6, 10, 16, 30, 32, 34.
*bfd*: 6, 10, 12, 14, 16, 18, 22, 24, 26, 28, 30, 32.
*bitpos*: 20.
*bits*: 20, 24.
*bitsize*: 20.
*byte_order*: 10.
*bytes*: 6, 10, 12, 16, 20, 26.
*bytes_len*: 6, 10, 12, 22, 26.
*calloc*: 10, 16, 26, 34.
*cap*: 6, 8, 12, 26, 32.
*cat*: 16.
*cat_end*: 6, 16, 32, 34.
*cat_start*: 6, 16, 32, 34.
*catalogue*: 2, 10.
*category*: 6, 12, 16, 30.
*chmod*: 32.
*class_value*: 14, 18, 26.
*close*: 32.
*cnt*: 28.
*common_addr*: 6, 16, 18, 30.
*common_align*: 6, 14, 16.
*common_size*: 6, 14, 16, 32.
*count*: 6, 8, 10, 12, 22, 26.
*cursor*: 16, 22, 28.
*d*: 8.
*data*: 20, 32, 34.
*data_offset*: 6, 26.
*data_size*: 6, 26.
*dataoff*: 26.
*datasize*: 26.
*db*: 6, 10.
*def_count*: 6, 10, 26.
*defs*: 6, 10, 26.
*digit*: 26.
*discard*: 6, 10, 12, 14, 26.
*done*: 34.
*dsym_buf_free*: 34.
*dsym_buf_init*: 34.
*dsym_buf_puts*: 34.
*dsym_diags_free*: 10, 26.
*dsym_diags_init*: 10, 26.
*DsymBuf*: 32, 34.
*DsymDiags*: 8, 10, 26.
*duplicate*: 12.
*e*: 28.
*e_ehsize*: 34.
*e_entry*: 34.
*e_ident*: 34.
*e_machine*: 34.
*e_phentsize*: 34.
*e_phnum*: 34.
*e_phoff*: 34.
*e_shentsize*: 34.
*e_shnum*: 34.
*e_shoff*: 34.
*e_shstrndx*: 34.
*e_type*: 34.
*e_version*: 34.
*eh*: 34.
`EI_CLASS`: 34.
`EI_DATA`: 34.
`EI_OSABI`: 34.
`EI_VERSION`: 34.
*element_size*: 8.
`ELFCLASS32`: 34.
`ELFCLASS64`: 34.
`ELFDATA2LSB`: 34.
`ELFMAG`: 34.
`ELFOSABI_SYSV`: 34.
*elf32*: 6, 10, 14, 18, 20, 26, 28, 30, 34.
*Elf32_Ehdr*: 34.
*Elf32_Phdr*: 34.
*Elf32_Shdr*: 34.

ELF32_ST_INFO: 34.
*Elf32_Sym*: 34.
*Elf64_Ehdr*: 34.
*Elf64_Half*: 34.
*Elf64_Phdr*: 34.
*Elf64_Shdr*: 34.
ELF64_ST_INFO: 34.
*Elf64_Sym*: 34.
EM_X86_64: 34.
EM_386: 34.
*encoded*: 24.
*end*: 34.
*entry*: 2, 34.
*entsize*: 22, 28.
*errno*: 32.
ET_EXEC: 34.
EV_CURRENT: 34.
*exists*: 28.
*explicit_addend*: 24.
*extracted*: 6, 26.
*f*: 10, 32, 34.
*false*: 8, 10, 12, 14, 16, 18, 22, 24, 26, 28, 30, 32, 34.
*fclose*: 10, 32.
*fd*: 32.
*fdopen*: 32.
*ferror*: 10, 32.
*fflush*: 32.
*fileno*: 32.
*flags*: 12.
*fmt*: 8.
*fopen*: 10, 32.
*format*: 10.
*found*: 24.
*fprintf*: 30.
*fread*: 10.
*free*: 10, 12, 26, 32, 34.
*fseek*: 10.
*fseeko*: 32.
*fsync*: 32.
*ftell*: 10.
*fwrite*: 32.
*g*: 14, 16, 18, 30, 32, 34.
*global_cap*: 6, 14.
*global_count*: 6, 10, 14, 16, 18, 30, 32, 34.
*globals*: 6, 10, 14, 16, 18, 30, 32, 34.
*gnu_names*: 26.
*got*: 6, 10, 16, 24, 28.
*got_base*: 6, 16, 18.
*got_cap*: 6, 28.
*got_count*: 6, 16, 24, 28.
*h*: 20, 26.
*has_byte_order*: 10.
*has_format*: 10.
*how*: 20, 24.
*i*: 10, 12, 14, 16, 18, 20, 22, 26, 28, 30, 32, 34.
*image*: 6, 10, 16, 24, 28, 30, 34.
*image_offset*: 6, 16, 24, 28.
*image_size*: 6, 16, 28, 30, 34.
*in*: 10, 12, 16, 18, 22, 24, 26, 28, 30.
*index*: 2, 18.
*info*: 12, 22, 28.
*input*: 6, 12, 14, 16, 18, 24, 28, 30, 32.
*input_cap*: 6, 10, 26.
*input_count*: 6, 10, 12, 14, 18, 22, 26, 28, 30.
*inputs*: 6, 10, 12, 14, 16, 18, 22, 24, 26, 28, 30, 32.
*int32_t*: 20.
INT64_MAX: 32, 34.
*int64_t*: 8, 12, 20, 22, 28.
*items*: 8.
*j*: 12, 14, 26, 28.
*k*: 12, 22, 24, 26, 28.
*l*: 26.
*labels*: 34.
*ld*: 2, 8, 10, 12, 14, 16, 18, 20, 22, 24, 26, 28, 30, 32, 34.
*ld_add_archive*: 10, 26.
*ld_align*: 8, 16.
*ld_align_up*: 32, 34.
*ld_apply_one*: 22, 24.
*ld_archive_member_add*: 26.
*ld_archive_number*: 26.
LD_BIND_GLOBAL: 4, 14, 26.
LD_BIND_LOCAL: 4, 14, 18, 26.
LD_BIND_WEAK: 4, 14, 18.
*ld_buf_append*: 32, 34.
*ld_category*: 12.
*ld_check_global_addresses*: 18, 30.
*ld_check_undefined*: 14, 30.
*ld_close_atomic*: 32, 34.
*ld_collect_globals*: 14, 30.
*ld_collect_got*: 28, 30.
*ld_collect_parts*: 12, 30.
*ld_defined_now*: 26.
*ld_definition_address*: 18, 30.
*ld_diagnostic*: 8, 10, 26.
LD_EM_X86_64: 4, 10, 26.
LD_EM_386: 4, 10, 26.
LD_ET_REL: 4, 10, 26.
*ld_extract_archives*: 26, 30.
*ld_fail*: 8, 10, 12, 14, 16, 18, 20, 22, 24, 26, 28, 30, 32, 34.
*ld_fill_got*: 28, 30.
*ld_global_address*: 18, 30, 32.
*ld_grow*: 8, 10, 12, 14, 26, 28.

*ld_how*: 20, 24.
*ld_load_le*: 20, 24.
*ld_lookup*: 14, 18.
LD_MAX_IMAGE: 4, 16, 34.
LD_MAX_INPUT: 4, 10.
*ld_open_atomic*: 32, 34.
*ld_place*: 16, 30.
*ld_power_two*: 8, 12, 14.
*ld_prepare_groups*: 12, 30.
*ld_put_relocated*: 20, 24.
*ld_read_bytes*: 10.
*ld_referenced_address*: 18, 24, 28.
*ld_relocate*: 22, 30.
*ld_seek*: 32, 34.
LD_SHF_ALLOC: 4, 12.
LD_SHF_EXEC: 4, 12.
LD_SHF_UNSUPPORTED: 4, 12.
LD_SHF_WRITE: 4, 12.
LD_SHN_ABS: 4, 14, 18, 32.
LD_SHN_COMMON: 4, 14.
LD_SHN_UNDEF: 4, 14, 18, 26.
LD_SHT_NOBITS: 4, 12, 16, 24.
LD_SHT_NOTE: 4, 12.
LD_SHT_PROGBITS: 4, 12.
LD_SHT_REL: 4, 22, 28.
LD_SHT_RELA: 4, 22, 28.
LD_SHT_SYMTAB: 4, 22.
*ld_sign_extend*: 20, 24.
*ld_span*: 8, 12, 22.
*ld_store_le*: 20, 28.
*ld_strdup*: 8, 10, 12, 14, 26.
*ld_symbol_section*: 32, 34.
*ld_validate_howto*: 20, 24.
*ld_wanted_now*: 26.
*ld_write_all*: 32, 34.
*ld_write_elf32*: 34.
**LDArchive**: 6, 10, 26.
**LDGlobal**: 6, 14, 16, 18, 30, 32, 34.
**LDGOT**: 6, 28.
**LDHow**: 20, 24.
**LDInput**: 6, 10, 12, 16, 18, 22, 24, 26, 28, 30.
**LDMember**: 6, 10, 26.
**LDPart**: 6, 12, 16, 18, 24, 30.
*len*: 8, 32, 34.
*length*: 6, 8, 10, 12, 16, 18, 24, 26, 30.
*link*: 22.
*linked*: 6, 10, 30, 32, 34.
*loops*: 26.
*m*: 10, 26.
*malloc*: 8, 10, 26, 32.
*max*: 20.
*max_file*: 34.
*me*: 10.
*members*: 6, 10, 26.
*memcmp*: 10, 26.
*memcpy*: 8, 16, 26, 32, 34.
*memset*: 14, 20, 26, 34.
*min*: 20.
*mkstemp*: 32.
*msg*: 8.
*n*: 8, 10, 12, 26, 32, 34.
*name*: 2, 6, 10, 12, 14, 18, 20, 24, 26, 30, 32, 34.
*name_len*: 26.
*namebuf*: 26.
*nameoff*: 34.
*names_size*: 26.
*nbytes*: 24.
*needed*: 8.
*no_op*: 20, 24.
*nsec*: 22, 28.
*number*: 20.
*obj*: 14, 26.
*object_path*: 2.
*off*: 8, 12, 26.
*off_t*: 32.
*offset*: 8, 12, 16, 22, 24, 32.
*ok*: 10, 12, 32, 34.
*old*: 8.
*out*: 8, 26, 30.
*p*: 8, 12, 16, 18, 30, 32.
*p_align*: 34.
*p_filesz*: 34.
*p_flags*: 34.
*p_memsz*: 34.
*p_offset*: 34.
*p_paddr*: 34.
*p_type*: 34.
*p_vaddr*: 34.
*part*: 18, 22, 24.
*part_cap*: 6, 12.
*part_count*: 6, 12, 16, 30.
*part_index*: 24.
*part_of_section*: 6, 10, 12, 18, 22, 26, 28.
*parts*: 6, 10, 12, 16, 18, 24, 30.
*patch*: 24.
*path*: 2, 6, 10, 12, 14, 18, 22, 24, 26, 30, 32, 34.
*pcrel*: 20, 24.
PF_R: 34.
PF_W: 34.
PF_X: 34.
*ph*: 34.
*place*: 24.
*pos*: 26.
*PRId64*: 12, 14, 20.

*PRIu64*: 8, 30.
*PRIx64*: 30.
*progress*: 26.
PT_GNU_STACK: 34.
PT_LOAD: 34.
*q*: 32.
*r*: 22, 24, 28.
*rank*: 6, 14, 16, 18, 30, 32, 34.
*realloc*: 8, 32.
*rec*: 28.
*record*: 22.
*rel*: 22.
*rela*: 22.
*relocations*: 20.
*remove*: 32.
*rename*: 32.
*rightshift*: 20.
*rs*: 22.
*rsec*: 22.
*s*: 8, 12, 14, 16, 18, 22, 24, 26, 30, 32.
*sec*: 12, 22, 28.
SEC_COUNT: 34.
*section*: 6, 12, 16, 24, 30.
SEEK_END: 10.
SEEK_SET: 10, 32.
*seen*: 12.
SEG_COUNT: 34.
SELFMAG: 34.
*sh*: 34.
*sh_addr*: 34.
*sh_addralign*: 34.
*sh_entsize*: 34.
*sh_flags*: 34.
*sh_info*: 34.
*sh_link*: 34.
*sh_name*: 34.
*sh_offset*: 34.
*sh_size*: 34.
*sh_type*: 34.
SHF_ALLOC: 34.
SHF_EXECINSTR: 34.
SHF_WRITE: 34.
SHN_ABS: 32.
SHN_UNDEF: 32.
*shnames*: 34.
*shndx*: 14, 18, 26, 32.
*shoff*: 34.
*shstr*: 34.
*shstroff*: 34.
SHT_NOBITS: 34.
SHT_PROGBITS: 34.
SHT_STRTAB: 34.
SHT_SYMTAB: 34.
*sig*: 12.
*signed_range*: 20, 24.
*size*: 2, 12, 14, 20, 22, 26, 28, 32.
SIZE_MAX: 8, 10, 16, 18, 22, 26, 28, 32.
*snprintf*: 26.
*spec*: 6, 10, 20, 22, 26, 28.
*sprintf*: 32.
*st*: 34.
*st_info*: 34.
*st_name*: 34.
*st_shndx*: 34.
*st_size*: 34.
*st_value*: 34.
*start*: 26, 34.
STB_GLOBAL: 34.
STB_WEAK: 34.
*storage*: 8.
*str*: 34.
*strcmp*: 10, 12, 14, 18, 26, 30.
*stream*: 2.
*strerror*: 32.
*strlen*: 8, 26, 32, 34.
*strncmp*: 26.
*stroff*: 34.
STT_NOTYPE: 34.
*sym*: 24.
*symbol*: 6, 14, 18, 24, 28, 30, 32.
*symbols*: 34.
*symcount*: 34.
*symtab*: 22.
*syoff*: 34.
*sz*: 32, 34.
*target*: 2, 10, 22.
*temp*: 32.
*tmp*: 34.
*top*: 20.
*total*: 8.
*true*: 8, 10, 12, 14, 16, 18, 20, 22, 24, 26, 28, 30, 32.
*type*: 12, 16, 22, 24, 28.
*uint16_t*: 34.
UINT32_MAX: 34.
*uint32_t*: 12, 32, 34.
UINT64_MAX: 8, 16, 18, 34.
*uint64_t*: 2, 6, 8, 10, 12, 14, 16, 18, 20, 24, 28, 30, 32, 34.
*uint8_t*: 2, 6, 10, 12, 16, 20, 24, 26, 30.
*unlink*: 32.
*v*: 26.
*va_end*: 8.
*va_start*: 8.
*value*: 14, 18, 20, 24, 30.

*vsnprintf*: 8.
*wanted*: 26.
*what*: 8.
*z*: 10, 12, 26, 28.
*zero*: 34.

⟨ Applying relocation fields 24 ⟩ Used in section 3.
⟨ COMDAT and section collection 12 ⟩ Used in section 3.
⟨ Checked arithmetic and diagnostics 8 ⟩ Used in section 3.
⟨ Executable ELF32 output 34 ⟩ Used in section 3.
⟨ Executable ELF64 output 32 ⟩ Used in section 3.
⟨ GOT synthesis 28 ⟩ Used in section 3.
⟨ Global definition selection 14 ⟩ Used in section 3.
⟨ Headers and ABI constants 4 ⟩ Used in section 3.
⟨ Input and global state 6 ⟩ Used in section 3.
⟨ Lazy archive extraction 26 ⟩ Used in section 3.
⟨ Lifecycle and input acquisition 10 ⟩ Used in section 3.
⟨ Link phases and diagnostics 30 ⟩ Used in section 3.
⟨ Placement and page boundaries 16 ⟩ Used in section 3.
⟨ Relocation section iteration 22 ⟩ Used in section 3.
⟨ Relocation validation 20 ⟩ Used in section 3.
⟨ Symbol address lookup 18 ⟩ Used in section 3.
⟨ `src/ArchLD.h` 2 ⟩

# ArchLD: An ArchBFD-driven Static Linker


| | Section | Page |
|---|---|---|
| The linker | 1 | 1 |
| The public C API | 2 | 2 |
| Translation unit | 3 | 3 |
| Headers and ABI constants | 4 | 4 |
| Input and global state | 6 | 5 |
| Checked arithmetic and diagnostics | 8 | 7 |
| Lifecycle and input acquisition | 10 | 9 |
| COMDAT and section collection | 12 | 12 |
| Global definition selection | 14 | 15 |
| Placement and page boundaries | 16 | 17 |
| Symbol address lookup | 18 | 19 |
| Relocation validation | 20 | 21 |
| Relocation section iteration | 22 | 23 |
| Applying relocation fields | 24 | 25 |
| Lazy archive extraction | 26 | 26 |
| GOT synthesis | 28 | 31 |
| Link phases and diagnostics | 30 | 32 |
| Executable ELF64 output | 32 | 34 |
| Executable ELF32 output | 34 | 37 |